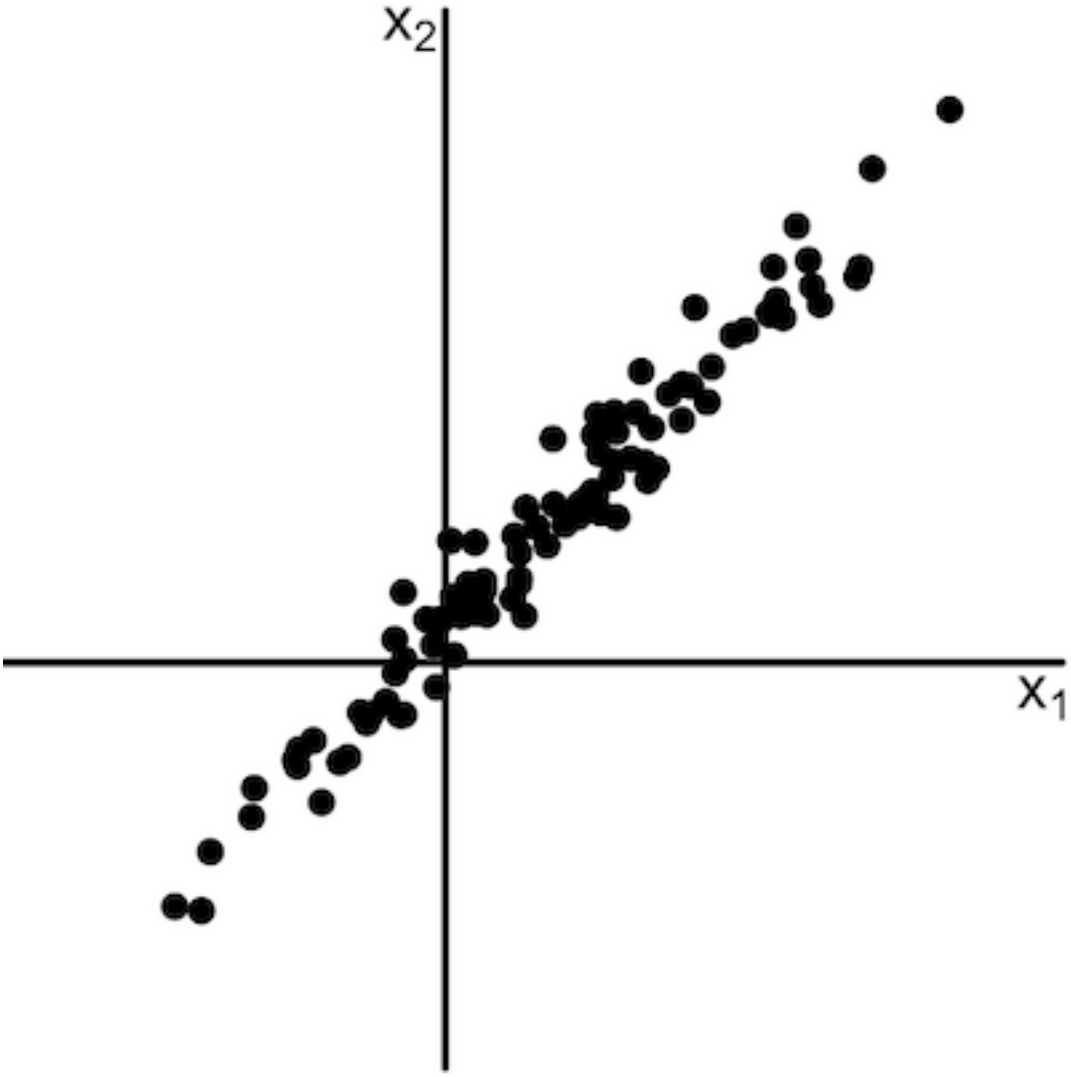
Part 15: Principal Components Analysis

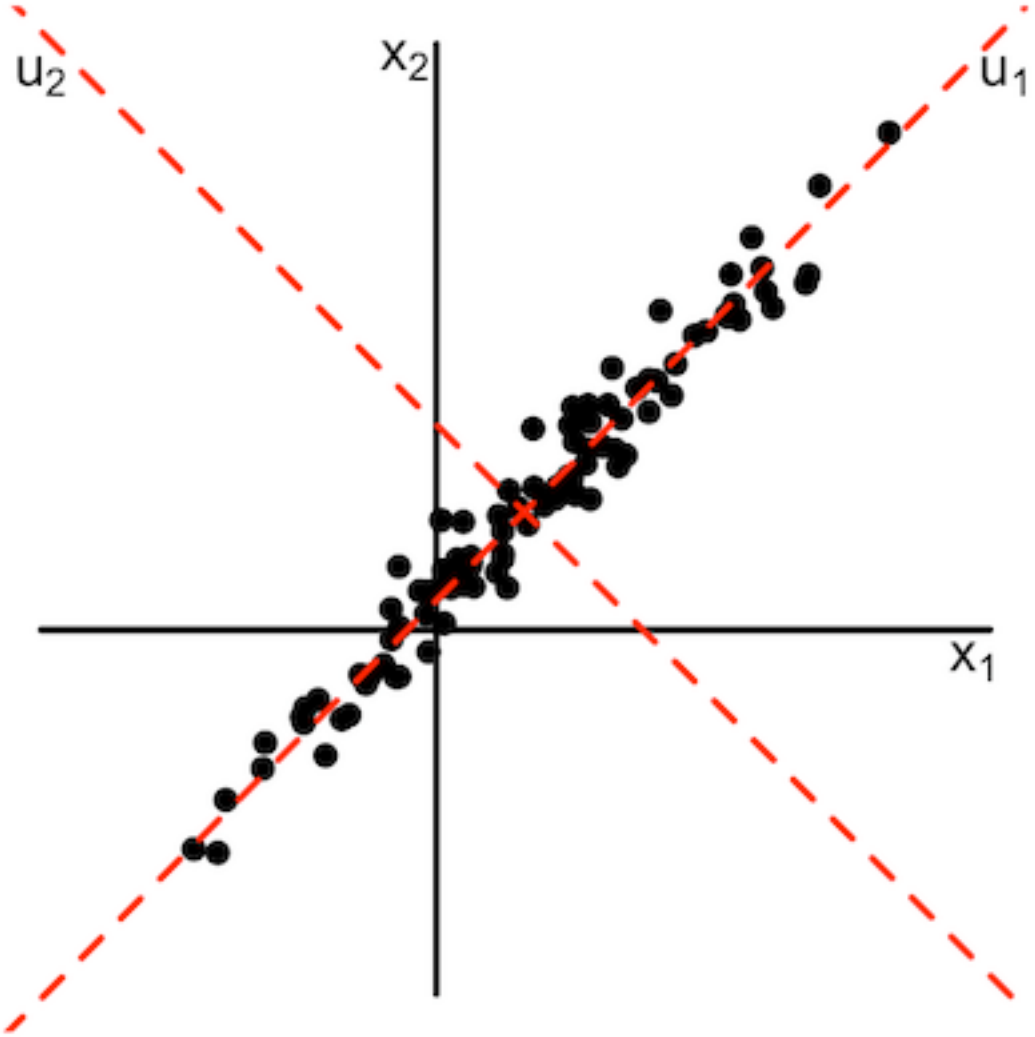
A standard, classic tool for dimension reduction is **principal components analysis (PCA)**. It has some drawbacks, mostly in that it is best suited to find **linear** structure in data, but in the right situation it can be a very effective tool.

First, consider the synthetic data shown in the following figure.



Exercise: Comment on the structure seen in the pair of variables shown in the previous figure. Would you say that the current pair of axes is the most "natural" way to represent the data?

The figure below shows the same data, with a new (u_1, u_2) coordinate system shown as the dashed lines.



These axes seem to be a more natural representation of the data. In particular, we note that the u_1 axis captures the major source of variability in the data. These axes are formed from the **principal components** of the data.

Formal Setup of PCA

Suppose we have observed n vectors $\mathbf{x}_i$, each of which is of dimension p:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T, \qquad i = 1, 2, \ldots, n.$$

For example, $\mathbf{x}_i$ could be a single yield curve, with $p \approx 10$.

Consider the collection of all possible linear combinations of the original variables formed by different choices of $\mathbf{u}$:

$$v_i = \sum_{j=1}^{p} u_j (x_{ij} - \overline{x}_j), \qquad i = 1, 2, \ldots, n$$

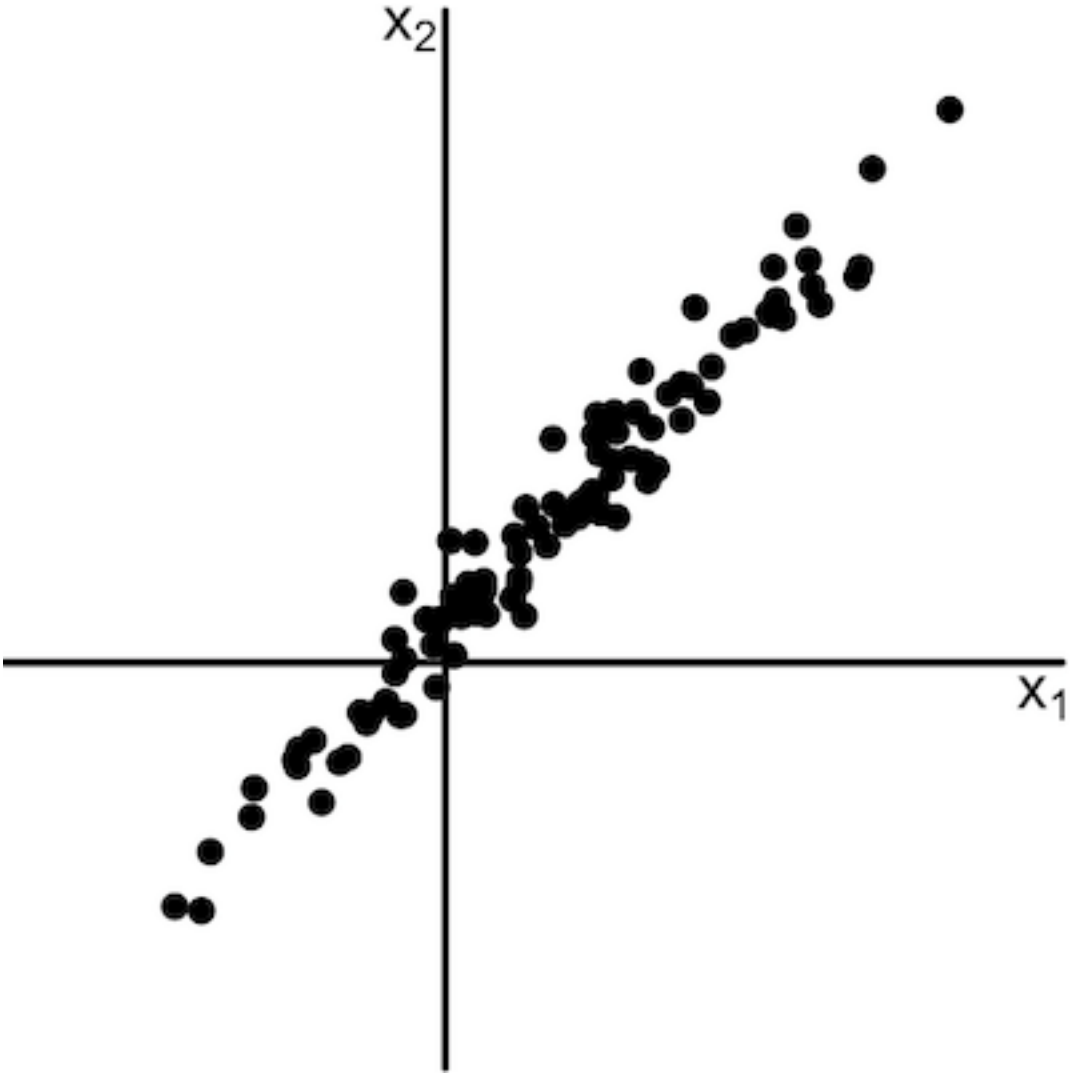
where $\overline{x}_j$ is the mean of the j^{th} variable, subject to

$$\sum_{j=1}^{p} u_j^2 = 1.$$

Hence, we can think of each choice of $\mathbf{u}$ as being a different **direction** centered on the sample mean.

The new measurements $v_1, v_2, \ldots, v_n$ are the **projections** of the original observations onto this new direction.

We now ask: What choice of $\mathbf{u}$ maximizes the sample variance of the resulting numbers $v_1, v_2, \ldots, v_n$?



The Math:

If $\mathbf{A}$ is a symmetric, positive definite matrix then the choice of $\mathbf{u}$ that maximizes $\mathbf{u}^T\mathbf{A}\mathbf{u}$ subject to $\mathbf{u}^T\mathbf{u}$ is the first eigenvector of $\mathbf{A}$.

Recall that

$$V(\mathbf{u}^T\mathbf{x}) = \mathbf{u}^T\Sigma\mathbf{u}$$

where $V(\mathbf{x}) = \Sigma$.

We approximate Σ using the sample covariance matrix $\widehat{\Sigma}$.

The leading eigenvector of $\widehat{\Sigma}$ is the first principal component.

The additional principal components $\mathbf{u}_2, \mathbf{u}_3, \ldots, \mathbf{u}_p$ are found by finding successive eigenvectors of $\widehat{\Sigma}$.

Some comments:

A new coordinate system is being constructed in which the p-dimensional data are represented. The center of this coordinate system is the sample mean of the data, but the axes are the principal components $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_p$.

When expressed in this new coordinate system, each of the dimensions have sample correlation of zero.

The amount of "variance explained" by each principal component is less than each of the previous principal components.

In many situations, we will choose some cutoff $q \ll p$ such that we will only retain the first q axes in our new coordinate system. This will be because these first q components "capture" most of the variability in the data.

For instance, in the toy example above, most of the variability in the data is "captured" by the first axis $\mathbf{u}_1$.

In typical applications p will be very large, and a low-dimensional representation will be of great value.

PCA on Yield Curves

Our previous discussion of yield curves suggested that these share a common shape, and that it does not require ten numbers to describe changes in this shape from day to day.

This is an ideal situation to consider dimension reduction. The simplicity of the shapes of the curves suggests that PCA is worth trying.

For this analysis we will use the yield curves from 2010 to the present. We need to extract the relevant rates to run the PCA. Note that there are 11 rates over this time period.

The command below takes a little longer to run.

```
In [1]: import pandas as pd

        fullYCweb = \
           pd.read_html("https://goo.gl/j97141")
        YCdata = fullYCweb[1]
```

Restrict to data since 2010.

```
In [2]: from datetime import datetime

        YCdata['Date'] = \
            YCdata['Date'].astype('datetime64')
        YCdata = YCdata[YCdata['Date'] > \
            datetime.strptime("2010-01-01", "%Y-%m-%d")]
```

We will remove the 2 month rates, as 8-week treasuries have only been around since October 2018.

```
In [3]: YCdata = YCdata.drop(['2 mo'], axis=1)
```

There are a couple rows with bad data that we will remove.

```
In [4]: print(YCdata[YCdata.drop("Date",axis=1).T.\
            astype(float).sum(axis=0) == 0])
        YCdata = YCdata[YCdata.drop("Date",axis=1).T.\
            astype(float).sum(axis=0) != 0]
```

```
           Date  1 mo  3 mo  6 mo  1 yr  2 yr  3 yr  5 yr  7 yr
5199 2010-10-11   NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN
6828 2017-04-14   0.0   0.0   0.0   0.0   0.0   0.0   0.0   0.0

      30 yr
5199    NaN
6828    0.0
```

Let's sample a few of the curves and create a plot of their yield curves.

First, use the `sample()` function to select ten at random.

```
In [5]: YCdata_sample = YCdata.sample(10)
```

Let's rename the columns so that the horizontal axis is scaled appropriately.

```
In [6]: YCdata_sample.\
            rename(columns={'1 mo': 1/12,
            '3 mo': 1/4, '6 mo': 1/2, '1 yr': 1,
            '2 yr': 2, '3 yr': 3, '5 yr': 5, '7 yr': 7,
            '10 yr': 10, '20 yr': 20, '30 yr': 30},
            inplace=True)
```

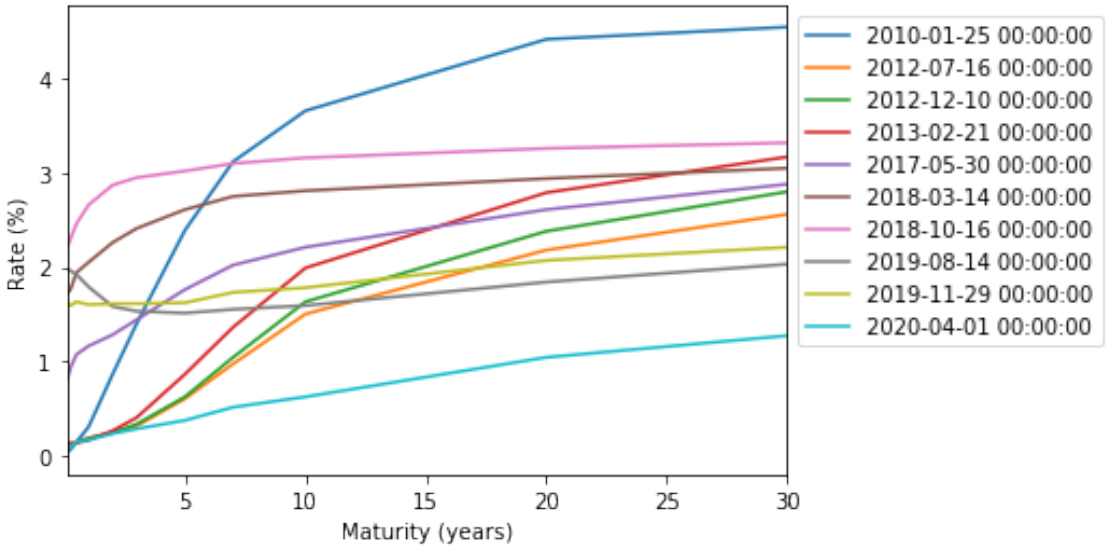
The data frame needs to be rearranged into a three-column format, with Date, Maturity, and Rate as the columns. This operation is generically referred to as **melting**.

```
In [7]: YCdata_melted = \
            pd.melt(YCdata_sample, id_vars='Date',
            value_vars=[1/12, 1/4, 1/2, 1, 2,
            3, 5, 7, 10, 20, 30], var_name='Maturity',
            value_name='Rate')
        YCdata_melted['Date'] = \
            YCdata_melted['Date'].astype('datetime64')
```

Finally, create the plot.

```
In [8]: import matplotlib.pyplot as plt

        ax = YCdata_melted.pivot("Maturity", "Date",
           "Rate").astype(float).plot()
        ax.ticklabel_format(axis='x', useOffset=False)
        plt.legend(bbox_to_anchor=(1.0, 1.0))
        plt.xlabel("Maturity (years)")
        plt.ylabel("Rate (%)")
        plt.show()
```



The PCA will be run on the day-to-day **shift** in the yield curve. The motivation is to characterize the low-dimensional structure in how the yield curve changes.

```
In [9]: YCshifts = YCdata.diff(1).dropna()
```

`scikit-learn` Learn has a PCA function.

```
In [10]: from sklearn.decomposition import PCA
```

The PCA is initialized using the following syntax. Note that you need to specify the number of components desired. At this point, this choice should be relatively large.

```
In [11]: pcaout = PCA(n_components=10)
```

Now run the PCA using `fit()`.

```
In [12]: pcaout.fit(YCshifts.drop('Date', axis=1))
         None
```

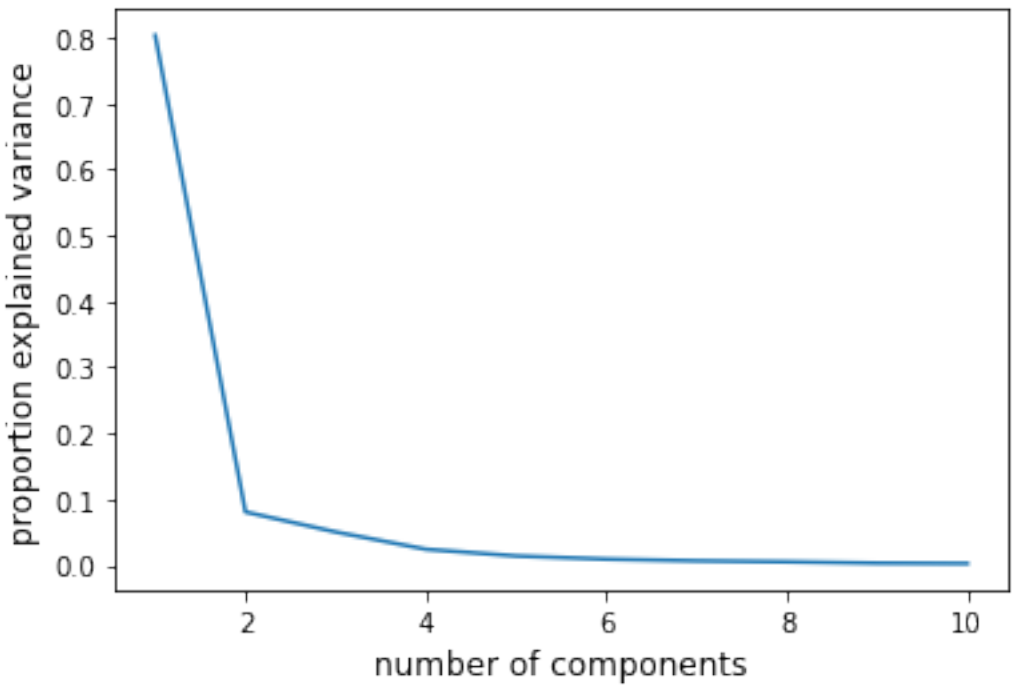
Exercise: What is the meaning of the argument `axis=1`?

We can see the proportion of the variance explained by each component. The plot created below is commonly referred to as the **scree plot**.

```
In [13]: pcaout.explained_variance_ratio_

Out[13]: array([0.80367424, 0.08058786, 0.05023836, 0.02398739,
                0.00901969, 0.00644045, 0.00498135, 0.00293534,
```

```
In [14]: plt.plot(range(1,11),
                  pcaout.explained_variance_ratio_)
         plt.xlabel('number of components', size=12)
         plt.ylabel('proportion explained variance', size=12)
         plt.show()
```



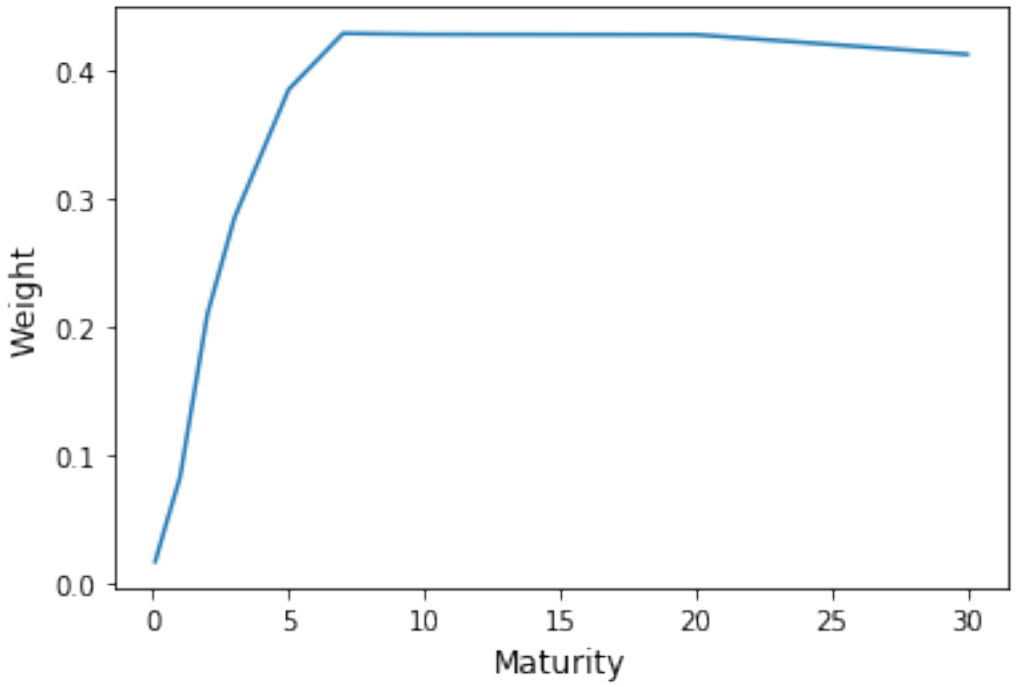
Exercise: Interpret the output above.

Here are the actual component vectors, sometimes called the **loadings**.

```
In [15]: pcaout.components_[0]

Out[15]: array([0.0166458 , 0.02872659, 0.04693629, 0.08245955,
                0.28423383, 0.38442732, 0.42804384, 0.42729944,
                0.41171032])

In [16]: Maturity = [1/12, 1/4, 1/2, 1, 2,
                     3, 5, 7, 10, 20, 30]
         plt.plot(Maturity, pcaout.components_[0])
         plt.xlabel('Maturity', size=12)
         plt.ylabel('Weight', size=12)
         plt.show()
```



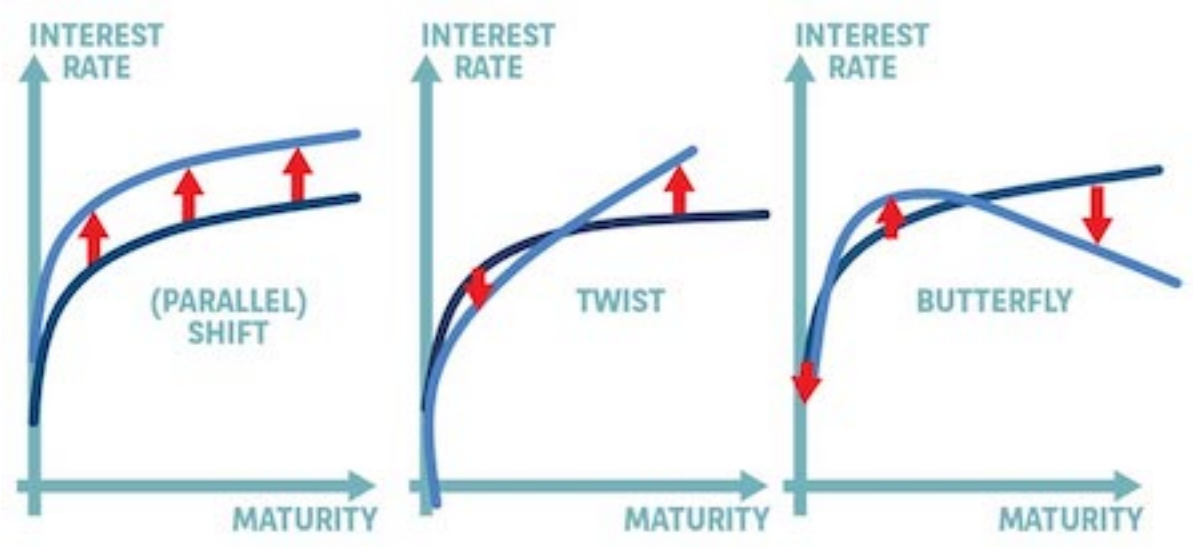


Figure from advisoranalyst.com

Exercise: Inspect the other loadings, and interpret the results.

The loadings give a **representation** of the shifts in yield curves. In fact, this is related to a common way to characterize how the yield curve has changed. The first three components are commonly given the names "parallel shift," "twist," and "butterfly."

The `transform()` function will project a new vector into this new representation. Note that Python is a little picky on the format.

```
In [17]: import numpy as np

         newshift = np.array([0.00, 0.00, 0.01, 0.01,
             0.01, 0.07, 0.05, 0.05, 0.03, 0.03, 0.04])
         pcaout.transform(newshift.reshape(1, -1))

Out[17]: array([[ 0.10839351, -0.02369142,  0.01219993, -0.0049
                  0.00340261,  0.0005116 ,  0.03775179,  0.0011
```

Scaling Variables for PCA

In cases where the variables being incorporated into PCA are on different scales, it is crucial that the variables be standardized prior to the analysis. It is customary to scale variables so that each has sample mean of zero, and sample variance of one.

This is equivalent to finding the eigenvectors of the correlation matrix instead of the covariance matrix. Python, by default, centers each variable so that they have mean zero. If you want the variables to be scaled, you should do that in advance of passing them on to `PCA()`.